

OWNER LAUNCH CHECKLIST

What remains before Duitlah can launch

Credentials, hosting, stores, legal operations, and integrations — separated into "configure now" versus "still requires engineering."

Current state

The application is feature-complete through V2 Phase 14 and runs locally. The remaining work is mainly production infrastructure, external provider accounts, store compliance, and a small number of deliberately unavailable integrations.

Your shortest path to a private beta

1. Buy/configure the production domain and choose the hosting providers.
2. Deploy PostgreSQL, Redis, S3-compatible storage, API, and web companion over HTTPS.
3. Set production secrets, including one Anthropic key for both AI and receipt OCR.
4. Implement real transactional email before inviting external testers.
5. Distribute the iPhone build through TestFlight and complete real-device acceptance testing.

Do not enable MyInvois in production yet

The HTTP adapter exists, but legally required certificate-based document signing is not implemented. Sandbox experiments are fine; production submission is not launch-ready.

1. Launch status at a glance

Area	Current state	What you must do
AI assistant	Real Anthropic adapter; disabled without key	Create key, set model, budget and test
Receipt OCR	Real Claude Vision adapter; manual fallback works	Same Anthropic key; test Malaysian receipts
Email	Logs messages and reset links only	Engineering + provider account required
Push notifications	Logs intent; no device-token pipeline	Engineering + APNs/FCM required
Subscriptions	Entitlements exist; store receipt verifier unavailable	Engineering + App Store/Play integration
MyInvois	Sandbox HTTP client; signing incomplete	LHDN credentials + certificate-signing engineering
Hosting	Docker/local configuration only	Deploy managed production stack
Mobile stores	Signed device build works	Store accounts, listings, compliance, release builds
Backups	Scripts and restore smoke test exist	Automate encrypted off-site backups and drills

Priority order

- P0 — domain, HTTPS hosting, database/storage, secrets, email, backups, legal review, TestFlight.
- P1 — Anthropic AI/OCR, monitoring, verified universal/app links, production runbooks.
- P2 — push notifications, paid subscriptions, MyInvois production signing, payment deeplinks.

2. Production hosting and domain

Recommended topology for the current architecture:

Component	Production requirement	Examples / notes
Domain + DNS	One HTTPS domain and API subdomain	duitlah.app + api.duitlah.app
Web companion	Next.js host	Vercel or equivalent
Spring Boot API	Persistent container runtime	Railway, Render, AWS, Fly.io, etc.
PostgreSQL	Managed PostgreSQL with PITR/backups	Do not expose publicly unless required
Redis	Managed Redis	Rate limits and ephemeral state only
Receipt storage	Private S3-compatible bucket	Presigned access; lifecycle/retention policy

Production DNS and TLS checklist

- ☐ Point the root/www domain at the web deployment.
- ☐ Point api.<domain> at the Spring Boot deployment.
- ☐ Enforce HTTPS and redirect HTTP to HTTPS.
- ☐ Set CORS_ALLOWED_ORIGINS and FRONTEND_URL to the exact HTTPS web origin.
- ☐ Set NEXT_PUBLIC_API_URL to the HTTPS API origin at web build time.
- ☐ Build mobile with API_BASE_URL and WEB_BASE_URL set to the production HTTPS origins.

Production container note

The repository's web Dockerfile runs the development server and is not the intended production web deployment. The API Dockerfile is the production-style image.

3. Secrets and environment configuration

Variable	Production value / decision
DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD	Managed PostgreSQL connection details; use a restricted application user
REDIS_HOST / REDIS_PORT	Managed Redis endpoint; add TLS/auth support if the provider requires it
JWT_SECRET	Generate at least 48 random bytes; never reuse the dev default
REFRESH_COOKIE_SECURE	true
REFRESH_COOKIE_SAME_SITE	Lax for same-site subdomains; None only with Secure for cross-site
REFRESH_COOKIE_DOMAIN	Set only if sharing across subdomains is necessary
CORS_ALLOWED_ORIGINS	Exact web origin(s), never * with credentials
FRONTEND_URL	Public HTTPS web origin
S3_ENDPOINT / S3_PUBLIC_ENDPOINT	Provider endpoints reachable by API and clients
S3_REGION / S3_BUCKET	Production region and private bucket
S3_ACCESS_KEY / S3_SECRET_KEY	Least-privilege bucket credentials
S3_PATH_STYLE_ACCESS	Usually false for AWS; provider-specific

Secret-handling rules

- ☐ Store secrets in the hosting provider's encrypted secret manager—not Git or mobile dart-defines.
- ☐ Use separate development, staging, and production credentials.
- ☐ Rotate JWT, database, storage, Anthropic, and provider keys on a defined schedule.
- ☐ Restrict team access and enable MFA on every provider account.

4. AI assistant and receipt OCR

One account, one key, two features

ANTHROPIC_API_KEY enables both the workspace-aware AI assistant and Claude Vision receipt parsing. Without it, the product remains honest: AI says unavailable and OCR opens manual review.

What you need to do

- ☐ Create an Anthropic organization/project and production API key.
- ☐ Set ANTHROPIC_API_KEY only on the backend host.
- ☐ Confirm ANTHROPIC_AI_MODEL and ANTHROPIC_OCR_MODEL are model identifiers available to your Anthropic account. The repository defaults to claude-sonnet-5.
- ☐ Set provider spending limits and alerts before inviting users.
- ☐ Test OCR against Malaysian receipts: SST, service charge, discounts, mixed languages, blurred images, and handwritten totals.
- ☐ Review Anthropic data-retention/privacy terms and reflect the processing in your privacy notice.
- ☐ Keep human confirmation mandatory; do not auto-post low-confidence OCR results.

Acceptance checks

- ☐ AI response reports available=true and tool results match deterministic backend calculations.
- ☐ Receipt scans populate merchant/date/amount/items and expose needsReview correctly.
- ☐ Provider outage or exhausted quota degrades cleanly without fake success.
- ☐ No API key appears in mobile/web bundles, logs, screenshots, or error payloads.

5. Integrations that still require engineering

Integration	Why a credential alone is insufficient	Next implementation
Transactional email	Only LoggingEmailSender exists; it prints verification/reset links to server logs	Implement provider adapter (SES, Postmark, Resend, etc.), templates, bounce handling
Push notifications	No device-token registration or APNs/FCM delivery	Token API + storage + APNs/FCM adapter + permission UX
Paid subscriptions	UnavailableSubscriptionVerifier is always wired	App Store/Play receipt verification, webhooks, restore purchases
MyInvois production	Certificate-based digital signing is missing	PKCS#12/key-store signing, sandbox certification, production endpoints
Payment handoff	Settlements provide manual instructions only	Optional DuitNow/e-wallet deeplinks; never claim funds moved
Telemetry	Generic webhook adapters exist, not a full vendor SDK	Configure webhook receiver or add Sentry/analytics adapter

MyInvois sandbox variables

- DUITLAH_MYINVOIS_ENABLED=true
- DUITLAH_MYINVOIS_CLIENT_ID and DUITLAH_MYINVOIS_CLIENT_SECRET
- DUITLAH_MYINVOIS_BASE_URL and DUITLAH_MYINVOIS_IDENTITY_URL (defaults are pre-production)
- DUITLAH_MYINVOIS_SCOPE=InvoicingAPI

Legal boundary

Do not point MyInvois at production until document signing is implemented and verified with LHDN. The current adapter may be used for sandbox development only.

6. Apple, Android, and verified links

Apple / TestFlight / App Store

- ☐ Maintain an active Apple Developer Program membership.
- ☐ Confirm bundle ID com.duitlah.duitlahMobile and production signing ownership.
- ☐ Create the App Store Connect app, SKU, age rating, privacy labels, screenshots, support URL, and privacy-policy URL.
- ☐ Create archive/release builds with production HTTPS dart-defines; do not ship the LAN address.
- ☐ Upload to TestFlight, add internal testers, then run external beta review if needed.
- ☐ Test Face ID, camera, photo library, QR scanning, deep links, account deletion, and offline/provider-failure states on physical devices.

Google Play

- ☐ Create the Play Console app for com.duitlah.duitlah_mobile.
- ☐ Create and securely back up the upload/release signing key; enable Play App Signing.
- ☐ Remove android:usesCleartextTraffic="true" for the production manifest.
- ☐ Complete Data safety, content rating, store listing, screenshots, support contact, and privacy-policy URL.
- ☐ Publish to internal testing before closed/open/production tracks.

Universal links / Android app links

- ☐ Host /.well-known/assetlinks.json with the Android package and release certificate SHA-256 fingerprint.
- ☐ Add the iOS Associated Domains entitlement (applinks:duitlah.app). It is not currently present.
- ☐ Host /.well-known/apple-app-site-association with the Apple Team ID, bundle ID, and /bagilah/join path.
- ☐ Verify links on physical iPhone and Android devices after HTTPS deployment.

7. Security, privacy, operations, and launch gate

Before real user data

- ☐ Have Malaysian counsel review Terms, Privacy Policy, PDPA obligations, tax disclaimers, and data-processing vendors.
- ☐ Replace support@duitlah.app placeholders with a working monitored mailbox.
- ☐ Define retention/deletion for receipts, logs, backups, abandoned accounts, and provider data.
- ☐ Add production email before enabling password reset for external users.
- ☐ Configure monitoring/alerts for API health, 5xx rate, database capacity, storage failures, queue/scheduler failures, and provider spend.
- ☐ Run dependency, container, and secret scanning in CI.

Backup and recovery

- ☐ Schedule encrypted database backups outside the primary failure domain.
- ☐ Enable point-in-time recovery where available.
- ☐ Back up object storage or enable versioning according to retention policy.
- ☐ Run scripts/postgres-restore-smoke-test.sh regularly against isolated infrastructure.
- ☐ Write recovery-time (RTO) and recovery-point (RPO) targets and conduct a restore drill.

Final go/no-go checks

- ☐ All backend, mobile, and web tests/builds pass from a clean checkout.
- ☐ Fresh database migrations apply successfully and rollback/recovery steps are documented.
- ☐ Production web/API health checks work from outside your network.
- ☐ Registration, verification, login, refresh, reset, deletion, receipt upload, Bagilah invitation, and settlement flows pass end to end.
- ☐ No UI claims that unavailable providers are connected or successful.
- ☐ TestFlight/Play internal testers approve the release candidate.

8. Suggested execution plan

Stage	Owner actions	Exit condition
1 • Foundation	Domain, provider accounts, managed DB/Redis/S3, hosting projects	All accounts protected by MFA; staging exists
2 • Deploy	Set secrets; deploy API/web; run migrations and restore test	HTTPS health and storage work externally
3 • Core launch services	Implement email; configure Anthropic and monitoring	Registration/reset/OCR/AI tested in staging
4 • Mobile beta	Production dart-defines, signing, TestFlight/internal Play	Physical-device acceptance passed
5 • Compliance	Legal review, store disclosures, support/runbooks	No unresolved P0 launch risk
6 • Public launch	Gradual rollout, alerts, backup schedule, incident owner	Stable metrics and support coverage
7 • Post-launch	Push, subscriptions, MyInvois signing, payment deeplinks	Each ships only after provider-specific testing

Concrete next three actions

1. Choose the domain and production hosting providers, then create a staging environment first.
2. Create the Anthropic project/key and a transactional-email provider account; email still needs an adapter implemented.
3. Create the App Store Connect record and ship the current build to internal TestFlight after replacing LAN URLs with staging HTTPS URLs.

Repository references

This guide was derived from .env.example, apps/api/.env.example, application.yml, docs/deployment.md, docs/security.md, docs/business-tax-and-einvoice.md, and the concrete provider implementations. Prepared 24 August 2026.